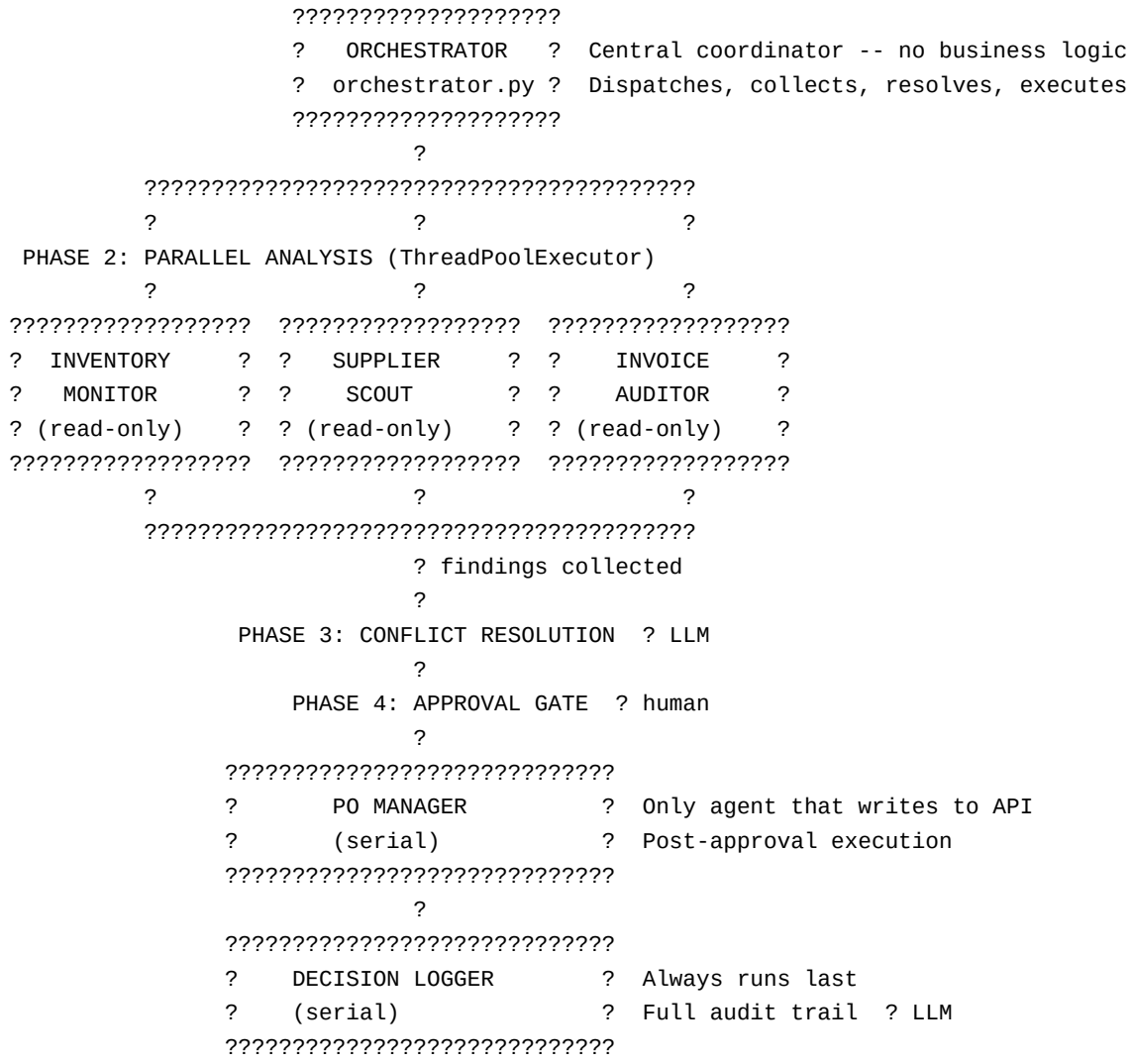
Betsy -- Orchestra Architecture

Parallel Analysis · Central Orchestrator · Explicit Conflict Resolution Specialist agents run concurrently. The orchestrator coordinates, resolves conflicts, and manages execution. LLM (local Ollama) is embedded in every agent's reasoning.

Overview



Entry Point -- `orchestra/run.py`

```
run_orchestra(
  triggered_by    = "manual" | "scheduler" | "test"
  api_base_url    = "http://localhost:8000"
  ollama_base_url = env:OLLAMA_BASE_URL  ->  default: http://localhost:11434
  ollama_model    = env:OLLAMA_MODEL     ->  default: mistral
  approval_mode   = "console" | "auto_approve" | "auto_reject" | "webhook"
  max_workers     = 4
)
-> OrchestraRun
```

Creates: `Orchestrator(client, llm, max_workers)` · calls `orchestrator.run(triggered_by)`

Message Types -- orchestra/schemas.py

All inter-component communication uses these types. No agent shares mutable state.

```
SituationBrief          Immutable snapshot of all API data for this run
  run_id · started_at · triggered_by
  inventory[] · suppliers[] · all_pos[] · open_pos[] · invoices[]
  active_scenario
  (Created once in Phase 1 -- never modified after)

AgentTask               Dispatched by orchestrator to one agent
  task_id · agent_name · task_type · brief (SituationBrief) · parameters · deadline

Finding                 One atomic observation from one agent
  finding_id · agent_name
  finding_type  ->  "stockout_risk" | "price_spike" | "duplicate_invoice" |
                    "supplier_unavailable" | "invoice_anomaly"
  severity      ->  "critical" | "warning" | "info"
  sku_id        ->  str | None
  confidence    ->  float 0.0-1.0  (from agent's LLM call)
  data          ->  {supporting numbers}
  llm_reasoning ->  str  (agent's LLM explanation)
  recommendation -> str  (what this agent thinks should happen)
  action_required -> bool

AgentResult             Full output from one agent for one task
  task_id · agent_name · success · findings[] · error | None · duration_ms · metadata

ConflictReport          Created when two agents have incompatible recommendations
  conflict_id · sku_id · agents_involved[]
  findings[]            (the conflicting findings)
  resolution            "highest_severity_wins" | "duplicate_blocks_po" | "human_escalation"
  resolved_finding      (the finding the orchestrator chose to act on)
  llm_resolution_reasoning  (orchestrator LLM explanation)
  resolution_reason     (short machine tag)

OrchestraDecision       Final orchestrator decision after conflict resolution
  decision_id · action
  source_finding · conflict_report | None
  requires_human · auto_approved
  reason · po_payload | None · flag_payload | None

OrchestraRun            Complete record of one full execution
  run_id · started_at · brief · tasks_dispatched[]
  agent_results[] · conflicts[] · decisions[] · actions_taken[]
  final_status · completed_at
```

Orchestrator -- orchestra/orchestrator.py

```
Role: Central coordinator. Holds no business logic.
      Builds brief -> dispatches agents -> collects findings ->
      resolves conflicts -> manages approvals -> triggers execution -> audits.

Class: Orchestrator(client, llm, max_workers=4, agent_timeout_s=10.0)
```

```
Methods
???????
run(triggered_by) -> OrchestraRun
  Phase 1: brief = _build_brief()
  Phase 2: results = _dispatch_parallel(brief)
  Phase 3: decisions, conflicts = _resolve(results)
  Phase 4: decisions = _process_approvals(decisions)
  Phase 5: actions = _execute(decisions, brief)
  Phase 6: _audit(run)  [always called, even on exception]

_build_brief() -> SituationBrief
  The only place API data is fetched in the entire orchestra.
  GET /inventory · /purchase-orders · /invoices · /suppliers · /scenario
  Constructs immutable SituationBrief passed to all agents.

_dispatch_parallel(brief) -> list[AgentResult]
  Submits InventoryMonitor, SupplierScout, InvoiceAuditor to ThreadPoolExecutor.
  Waits for all with timeout=agent_timeout_s (10s default).
  Timed-out agent -> AgentResult(success=False, error="Agent timed out", findings=[])
  Returns: all AgentResult objects (including failed ones -- orchestrator handles gracefully)

_resolve(results) -> (list[OrchestraDecision], list[ConflictReport])
  Aggregates all Finding objects from all successful AgentResults.
  Groups findings by sku_id.
  For each group with 2+ findings: calls _detect_conflicts() then _apply_precedence().
  Produces one OrchestraDecision per action required.

_process_approvals(decisions) -> list[OrchestraDecision]
  For each requires_human=True decision:
    gate.request_approval(decision) -> ApprovalResult
    approved  -> decision.auto_approved = True  + reviewer metadata
    rejected  -> decision.auto_approved = False + rejection note

_execute(decisions, brief) -> list[dict]
  POManagerAgent runs SERIALLY (not in thread pool -- no parallel writes).
  auto_approved generate_po -> POManagerAgent.execute_create_po(decision)
  all other decisions -> logged as pending/no-write

_audit(run) -> None
  DecisionLoggerAgent.write_run(run)  [always called in finally block]
```

Conflict Resolution Table

```
Conflict (Finding A + Finding B on same sku_id)
????????????????????????????????????????????????????????????????????????????????

  PRECEDENCE OVERRIDES  (code-level safety net applied before LLM)

  duplicate_invoice + stockout_risk
    Winner: duplicate_invoice
    Reason: Never create a PO while an invoice is under fraud investigation.
    Decision: flag_duplicate (PO is blocked entirely)

  price_spike + stockout_risk
```

```
    Winner: price_spike
    Reason: Ordering into a price spike requires human sign-off.
    Decision: flag_for_approval (human sees both urgency AND price context)

  supplier_unavailable + stockout_risk
    Winner: stockout_risk (with fallback supplier)
    Reason: Stockout survives; use next-best available supplier.
    Decision: generate_po with fallback supplier if one exists; else escalate

  price_spike + duplicate_invoice
    Winner: duplicate_invoice
    Reason: Duplicate flag takes absolute priority.

  No matching rule
    -> LLM call to orchestrator conflict prompt (see below)
    Fallback if LLM unavailable: higher severity wins -> higher confidence wins -> escalate

CONFLICT RESOLUTION LLM PROMPT
  SYSTEM: "You are a procurement decision orchestrator. Return JSON only:
           {winning_finding_id, resolution, reasoning}
           Safety rules you MUST honor:
           - Duplicate invoice always blocks a PO on the same SKU
           - Price spike >18% always requires human approval before ordering"
  USER:   "Conflicting findings for SKU {sku_id}:
           Finding A ({agent_a}): {data}
           Finding B ({agent_b}): {data}
           Which takes priority and why?"

ConflictReport written for EVERY conflict -- even when rule is clear.
Logged to /api/agent-log so dashboard can surface them.
```

Agent 1 -- Inventory Monitor (orchestra/agents/inventory_monitor.py)

```
Role:         Monitor stock levels. Identify critical shortages. Recommend order qty.
              Runs in parallel with SupplierScout and InvoiceAuditor.

Input:        AgentTask (reads brief.inventory, brief.open_pos only)
Output:       AgentResult with findings[]

API calls:    NONE -- reads brief only (thread-safe, no shared mutable state)
LLM:          YES -- urgency assessment and quantity recommendation

Functions
?????????
execute(task: AgentTask) -> AgentResult

  _find_critical_items(inventory, open_pos) -> list[dict]
    Filter: current_stock < reorder_point
    Exclude: SKUs that already have an open PO (avoids double-ordering)
    Compute: days_remaining = current_stock / daily_usage_avg
             stock_ratio    = current_stock / reorder_point
             urgency_score  = 1.0 / max(days_remaining, 0.1)
    Sort:    descending urgency_score
```

```
  _compute_order_qty(item) -> int
    qty = max_stock - current_stock
    fallback: 2 × reorder_point

  _build_findings(critical_items, llm) -> list[Finding]
    For each critical item:
      LLM call:
        SYSTEM: "You are a warehouse analyst. Return JSON only:
                 {urgency: critical|high|medium|low, recommended_qty: int,
                  confidence: float, reasoning: str}"
        USER:   "SKU: {sku_id} ({name})
                 Current stock: {n} | Reorder point: {n} | Max stock: {n}
                 Daily usage: {n} units/day | Days remaining: {n}
                 Open POs for this SKU: {count}"
      -> Finding(
          finding_type = "stockout_risk"
          severity     = "critical" if days_remaining < 2.0 else "warning"
          confidence   = from LLM
          data         = {days_remaining, urgency_score, recommended_qty, current_stock, reorder_point}
          llm_reasoning = LLM response
          recommendation = "generate_po"
          action_required = True
        )
    Fallback: severity from days_remaining rule, confidence = min(urgency_score/10, 1.0)
```

Agent 2 -- Supplier Scout (orchestra/agents/supplier_scout.py)

```
Role:         Evaluate all suppliers. Detect price spikes. Identify availability gaps.
              Builds the supplier matrix used by orchestrator for PO payloads.
              Runs in parallel with InventoryMonitor and InvoiceAuditor.

Input:        AgentTask (reads brief.suppliers, brief.inventory, brief.all_pos)
Output:       AgentResult with findings[]

API calls:    NONE -- reads brief only
LLM:          YES -- supplier ranking explanation + red flag identification

Functions
?????????
execute(task: AgentTask) -> AgentResult

  _score_supplier_for_sku(supplier, sku_id) -> float | None
    score = reliability_score / lead_days    (from tests/scenario_runner.py)
    Returns None if: availability=False OR sku_id not in supplier.catalog

  _build_supplier_matrix(brief) -> dict[str, list[dict]]
    {sku_id: [{supplier_id, score, unit_price, lead_days, available, reliability_score}]}
    sorted descending by score per SKU
    Stored in each Finding's data dict -- orchestrator uses it to build PO payloads

  _detect_price_spikes(brief) -> list[Finding]
    For each SKU in inventory:
      baseline   = inventory[i].unit_cost_avg
```

```
      best_quote = min(available quotes from supplier_matrix[sku_id])
      spike if   best_quote > baseline × 1.18
    -> Finding(
        finding_type  = "price_spike"
        severity      = "warning"
        confidence    = 0.0  (always requires human -- hardcoded)
        data          = {sku_id, best_quote, baseline, pct_above, threshold=0.18}
        recommendation = "flag_for_approval"
        action_required = True
      )

  _detect_supplier_unavailability(brief) -> list[Finding]
    For each SKU below reorder_point:
      if ALL suppliers are unavailable (or SKU not in any catalog):
        -> Finding(finding_type="supplier_unavailable", recommendation="escalate")

  _build_findings(brief, llm) -> list[Finding]
    Combines: price_spike findings + supplier_unavailability findings
    For stockout SKUs with available suppliers:
      LLM call:
        SYSTEM: "You are a supplier evaluation specialist. Return JSON only:
                 {recommended_supplier_id, confidence, tradeoff_summary, red_flags: []}"
        USER:   "SKU needed: {sku_id}
                 Urgency: high (days remaining: {n})
                 Available suppliers:
                 {json(supplier_matrix[sku_id])}"
      -> attaches LLM recommendation + red_flags to supplier_matrix in Finding.data
    Fallback: top-scored supplier, no red flags
```

Agent 3 -- Invoice Auditor (orchestra/agents/invoice_auditor.py)

```
Role:         Detect duplicate invoices and billing anomalies.
              Assess fraud likelihood using LLM.
              Runs in parallel with InventoryMonitor and SupplierScout.

Input:        AgentTask (reads brief.invoices, brief.all_pos)
Output:       AgentResult with findings[]

API calls:    NONE -- reads brief only
LLM:          YES -- fraud vs billing-error assessment

Functions
?????????
execute(task: AgentTask) -> AgentResult

  _find_duplicates(invoices) -> list[dict]
    Algorithm (matches server/routers/invoices.py _find_duplicates):
      Group invoices by (supplier_id, amount)
      For each group with 2+ invoices:
        Flag pairs where abs(date_diff) ? 60 days
        risk_level = "HIGH" if days_apart ? 30 else "MEDIUM"
    Returns: [{invoice_1, invoice_2, amount, days_apart, risk_level}]
```

```
  _detect_invoice_anomalies(invoices, all_pos) -> list[dict]
    For each invoice with a matching PO id:
      pct_diff = (invoice.total_amount - po.total_amount) / po.total_amount
      Flag if abs(pct_diff) > 0.20  (>20% deviation)
    Returns: [{invoice_id, po_id, invoice_amount, po_amount, pct_diff}]

  _build_findings(duplicates, anomalies, llm) -> list[Finding]
    For each duplicate pair:
      LLM call:
        SYSTEM: "You are a financial auditor. Return JSON only:
                 {risk_level: HIGH|MEDIUM|LOW, confidence: float,
                  fraud_likelihood: suspicious|likely_error|unknown,
                  reasoning: str}"
        USER:   "Duplicate invoice pairs:
                 {json(pairs)}
                 Are these billing errors or potential fraud?
                 Consider: time gap, amounts, supplier history."
      -> Finding(
          finding_type  = "duplicate_invoice"
          severity      = "warning"
          confidence    = 1.0 if risk=HIGH else 0.7
          sku_id        = invoice.sku_id if determinable
          data          = {invoice_ids, amount, days_apart, risk_level, fraud_likelihood}
          llm_reasoning = LLM response
          recommendation = "flag_duplicate"
          action_required = True
        )
    For each anomaly:
      -> Finding(finding_type="invoice_anomaly", confidence=0.8, recommendation="flag_anomaly")
    Fallback: confidence from risk_level rule only, fraud_likelihood="unknown"
```

Agent 4 -- PO Manager (orchestra/agents/po_manager.py)

```
Role:         The ONLY agent that makes API write calls.
              Runs SERIALLY after all analysis is complete and approvals are obtained.
              Validates every payload before writing -- prevents double-orders and bad data.

Called by:    orchestrator._execute() -- NOT dispatched to thread pool

API calls:    POST /api/purchase-orders
              PATCH /api/purchase-orders/{id}/status

LLM:          NONE -- pure validation + execution

Functions
?????????
execute_create_po(decision: OrchestraDecision) -> dict

  _validate_before_create(payload, brief) -> list[str]
    Checks (returns list of error strings -- empty = proceed):
      v supplier_id exists in brief.suppliers
      v supplier availability = True
      v sku_id exists in brief.inventory
```

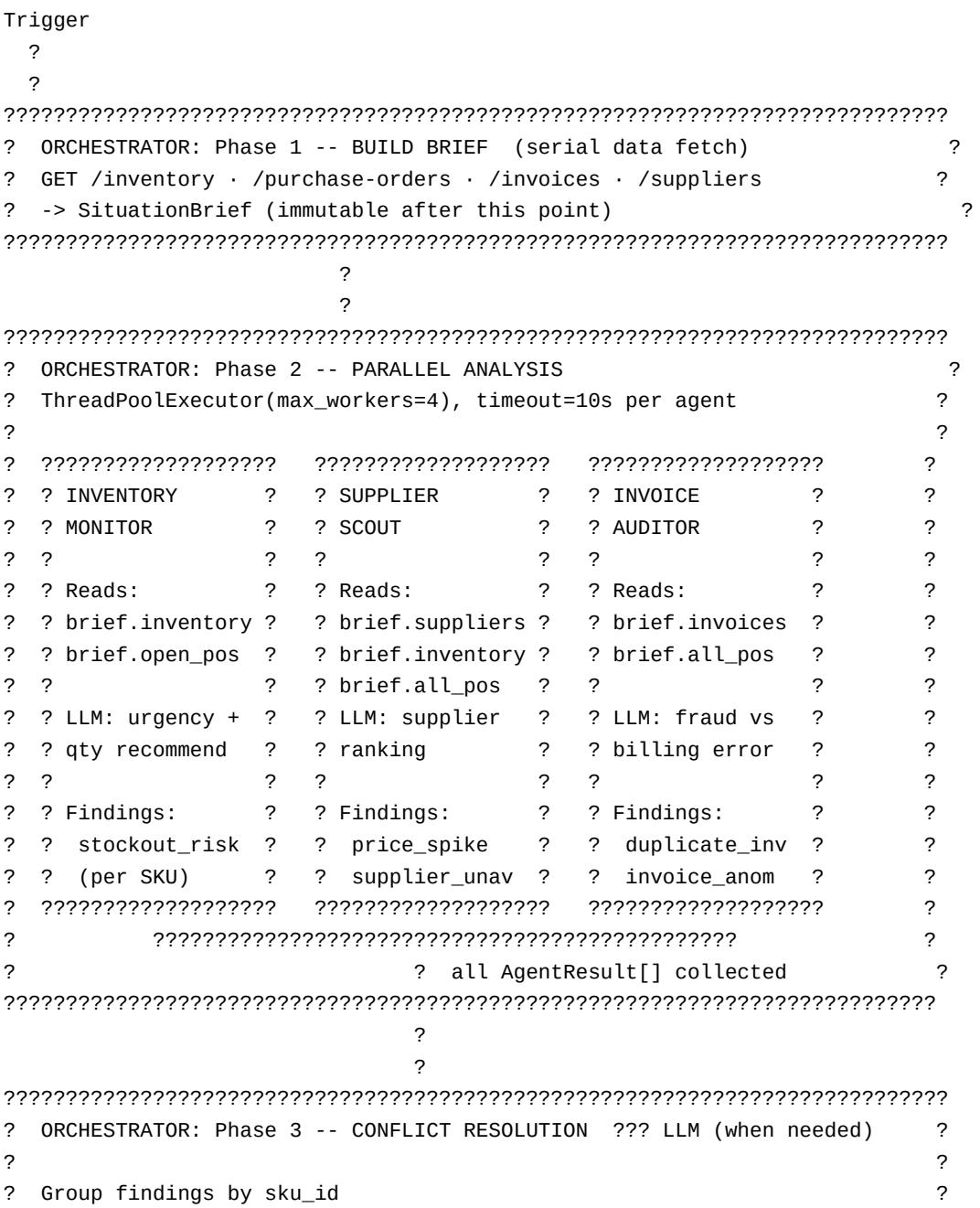
```
      v quantity > 0 AND quantity ? max_stock
      v unit_price > 0
      v no open PO already exists for this sku_id in brief.open_pos
    If validation errors: returns them, no API call made

  _build_po_payload(decision, brief) -> dict
    Extracts from decision.source_finding.data.supplier_matrix:
      supplier_id, sku_id, unit_price
    Computes: quantity = max_stock - current_stock
    Sets:     reason = decision.reason
              requested_by = "betsy-orchestra"

  API sequence (on validation pass):
    POST /api/purchase-orders  -> {po_id, status: "pending_approval", ...}
    PATCH /api/purchase-orders/{po_id}/status -> "approved"
    -> returns {decision_id, po_id, supplier_id, sku_id, quantity, success: True}

  On validation failure or API error:
    -> returns {decision_id, success: False, errors: [...]}
    (Execution failure is logged -- orchestra does NOT halt)

get_open_po_for_sku(sku_id, open_pos) -> dict | None
  Returns existing open PO for this SKU if found, None otherwise.
  Used by _validate_before_create to prevent double-ordering.
```

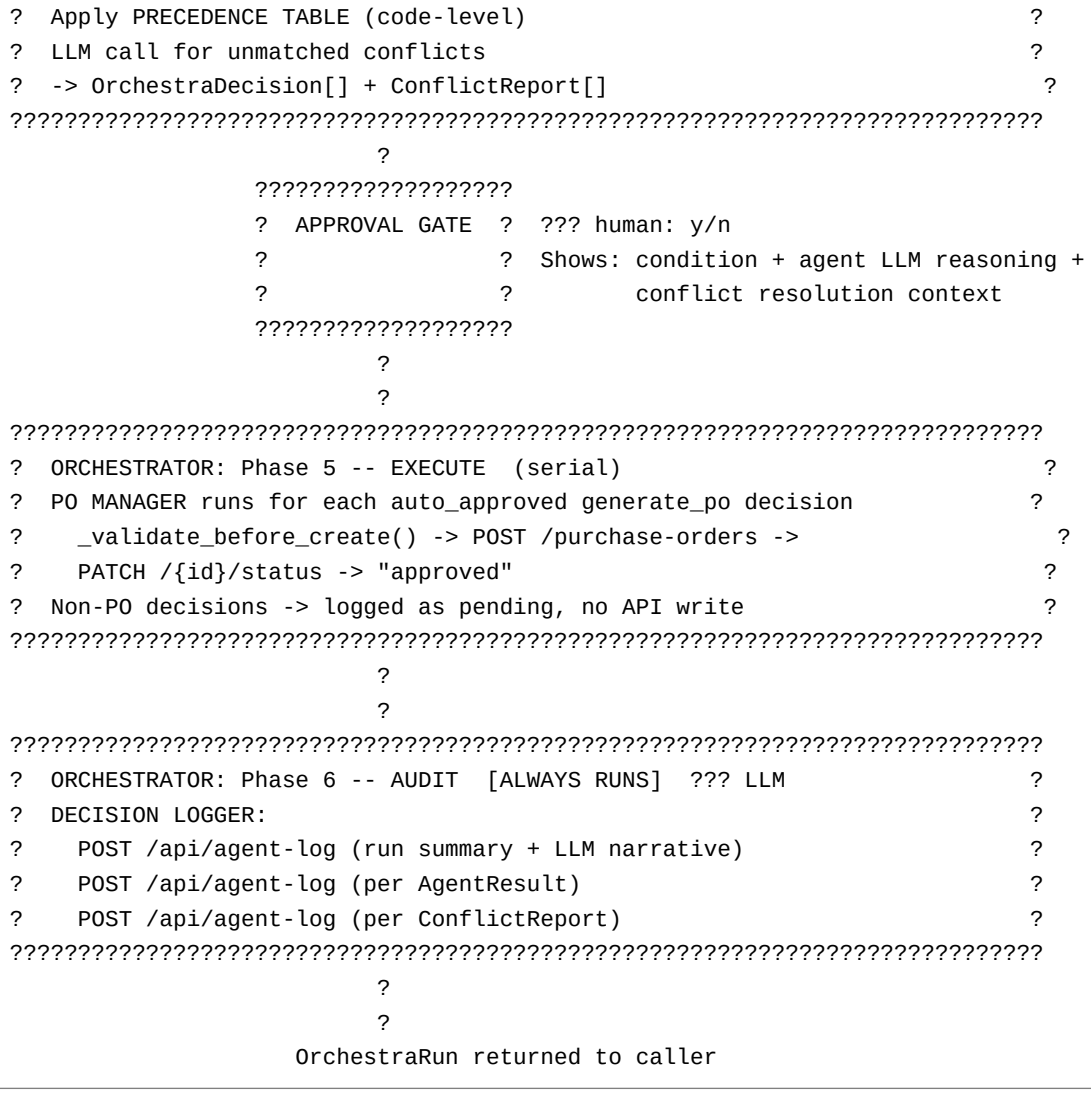
Agent 5 -- Decision Logger (orchestra/agents/decision_logger.py)

```
Role:         Write the complete OrchestraRun to /api/agent-log.
              Runs last, always -- even if other agents failed.
              Produces richer audit trail than pipeline: per-agent + per-conflict entries.

Called by:    orchestrator._audit() -- in finally block (always executes)

API calls:    POST /api/agent-log  (multiple entries)
LLM:          YES -- plain-English run narrative

Functions
?????????
write_run(run: OrchestraRun) -> None

  _format_run_summary(run, llm) -> dict
    LLM call:
      SYSTEM: "Summarize this procurement agent run in 2-3 sentences.
               Include: agents that ran, conditions found, conflicts resolved, actions taken."
      USER:   "Agents: {n} | Findings: {total} | Conflicts: {n} |
               Decisions: {n} | Actions executed: {n} | Status: {final_status}"
    POST /api/agent-log:
      {trigger:    "orchestra_run:{run_id}",
       analysis:   "{n} agents ran, {m} findings, {k} conflicts resolved",
       decision:   "{x} actions taken, {y} pending human review",
       confidence: weighted avg confidence across all decisions,
       metadata:   {full run summary + llm_narrative,
                    agent_results[] (per-agent summaries),
```

```
                    conflicts[] (per-conflict reports),
                    decisions[] (final decision list)}}

  _format_agent_findings(result: AgentResult) -> dict
    One /api/agent-log entry per AgentResult.
    Enables dashboard filtering by agent name.

  _format_conflict_reports(conflicts: list[ConflictReport]) -> list[dict]
    One /api/agent-log entry per ConflictReport.
    Each entry includes: agents_involved, resolution rule applied, llm_resolution_reasoning.
    Critical for audit transparency -- human reviewer can see exactly what conflicted.

  Fallback: if LLM unavailable -> structured JSON summary without narrative
```

Full Data Flow





LLM Integration Summary

Agent / Phase	LLM call	Prompt goal	Fallback
Inventory Monitor	Urgency assessment + qty recommendation	Assess criticality, recommend how muc
Supplier Scout	Supplier ranking + red flags	Explain tradeoffs, identify concerns	Rule: reli
Invoice Auditor	Fraud likelihood assessment	Distinguish billing error from fraud	Rule: conf
Orchestrator (Phase 3)	Conflict resolution	Pick winning finding and explain	Rule: higher se
Decision Logger (Phase 6)	Run narrative	2-3 sentence plain-English summary	Structured JSON

All LLM calls: `temperature=0.1`. JSON responses `try/except` parsed. Fallback always available.

Key Architecture Properties

```
Write isolation:
  Only POManagerAgent makes API write calls.
```

```
  All analysis agents (InventoryMonitor, SupplierScout, InvoiceAuditor) are read-only.
  Eliminates race conditions from parallel agent execution.

Conflict visibility:
  Every conflict produces a ConflictReport -- logged to /api/agent-log.
  Human reviewer can see exactly what conflicted and how it was resolved.
  Pipeline has implicit conflict handling; orchestra makes it explicit and auditable.

Extensibility:
  Add a new domain agent -> create agents/new_agent.py
  Register in orchestrator._dispatch_parallel()
  No changes to existing agents or schemas needed.

Agent timeout safety:
  Each parallel agent has a 10-second deadline.
  Timed-out agent -> AgentResult(success=False, findings=[])
  Orchestrator proceeds with available findings -- does not halt.
```

Portability

```
# Install Ollama: https://ollama.com/download
ollama pull mistral

# Override defaults via env vars (no code changes needed)
export OLLAMA_BASE_URL=http://localhost:11434   # or any remote Ollama host
export OLLAMA_MODEL=mistral                     # or llama3.1, phi3, etc.

# Run
python -m orchestra.run
python -m orchestra.run --approval-mode auto_approve   # for testing

# Run against a remote Ollama instance
OLLAMA_BASE_URL=http://192.168.1.50:11434 python -m orchestra.run
```

Uses Ollama's OpenAI-compatible `/v1/chat/completions` endpoint. Swapping to any OpenAI-compatible server = change `OLLAMA_BASE_URL` only.

Test Scenario Coverage

```
Scenario  |  Agent findings  |  Conflict  |  Decision  |  Expected
-------------------------------------------------------------------
stockout_warning  |  InventoryMonitor: stockout_risk critical SKU-003. SupplierScout: SUP-003 best (sco
-------------------------------------------------------------------------------
price_spike  |  InventoryMonitor: stockout_risk SKU-003. SupplierScout: price_spike SKU-003.  |  Confli
-------------------------------------------------------------------------------
duplicate_invoice  |  InvoiceAuditor: 3 duplicate findings, LLM says "likely billing error."  |  None
-------------------------------------------------------------------------------
supplier_oos  |  InventoryMonitor: stockout_risk SKU-003. SupplierScout: SUP-004 excluded, SUP-003 next
-------------------------------------------------------------------------------
```